



СибГУТИ

СИБИРСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ
ТЕЛЕКОММУНИКАЦИЙ
И ИНФОРМАТИКИ



Кафедра
ВЫЧИСЛИТЕЛЬНЫХ
СИСТЕМ

Векторизация в LLVM одного класса циклов методом их расщепления

Кулик Павел Евгеньевич

студент 4 курса

Институт информатики и вычислительной техники
Сибирский государственный университет телекоммуникаций и информатики

pashakulik58@gmail.com

Научный руководитель — д.т.н. профессор Курносов М.Г.

Векторизация

Векторизация — это процесс написания или трансляции программного кода с учётом использования векторных инструкций целевой платформы.

- Позволяет снизить время выполнения программы за счёт параллельной обработки данных;
- Векторизованная программа задействует векторные регистры процессора и не требует больше одного потока;
- Может быть осуществлена компилятором, что не требует изменений в исходном коде.

Векторизуемые таргеты LLVM

Таргеты, для которых LLVM может генерировать векторные инструкции:

AArch64

SPIRV

ARM

Hexagon

Mips

PowerPC

NVPTX

WebAssembly

RISCV

Spark

AMDGPU

SystemZ

VE

X86

Xtensa

Проблема

- Ни один из существующих на данный момент компиляторов не может векторизовать все циклы;
- Из тестового набора Extended Test Suite For Vectorizing Compilers (ETSVC), содержащего 151 тест, проверяющий способность компиляторов векторизовать циклы, clang version 22.0.0git может векторизовать только 103;
- Векторизация не всегда приводит к снижению времени выполнения программы.

Решение

- Продолжать улучшать компиляторы, расширяя их возможности

Исследуемые случаи

```
real_t s291(struct args_t * func_args) {  
    initialise_arrays(__func__);  
    gettimeofday(&func_args->t1, NULL);  
    int im1;  
    for (int nl = 0; nl < 2*iterations; nl++) {  
        im1 = LEN_1D-1;  
        for (int i = 0; i < LEN_1D; i++) {  
            a[i] = (b[i] + b[im1]) * (real_t).5;  
            im1 = i;  
        }  
        dummy(a, b, c, d, e, aa, bb, cc, 0.);  
    }  
    gettimeofday(&func_args->t2, NULL);  
    return calc_checksum(__func__);  
}
```

```
real_t s292(struct args_t * func_args) {  
    initialise_arrays(__func__);  
    gettimeofday(&func_args->t1, NULL);  
    int im1, im2;  
    for (int nl = 0; nl < iterations; nl++) {  
        im1 = LEN_1D-1;  
        im2 = LEN_1D-2;  
        for (int i = 0; i < LEN_1D; i++) {  
            a[i] = (b[i] + b[im1] + b[im2]) * (real_t).333;  
            im2 = im1;  
            im1 = i;  
        }  
        dummy(a, b, c, d, e, aa, bb, cc, 0.);  
    }  
    gettimeofday(&func_args->t2, NULL);  
    return calc_checksum(__func__);  
}
```

Разберёмся подробнее

```
real_t s291(struct args_t * func_args) {  
    initialise_arrays(__func__);  
    gettimeofday(&func_args->t1, NULL);  
    int im1;  
    for (int nl = 0; nl < 2*iterations; nl++) {  
        im1 = LEN_1D-1;  
        for (int i = 0; i < LEN_1D; i++) {  
            a[i] = (b[i] + b[im1]) * (real_t).5;  
            im1 = i;  
        }  
        dummy(a, b, c, d, e, aa, bb, cc, 0.);  
    }  
    gettimeofday(&func_args->t2, NULL);  
    return calc_checksum(__func__);  
}
```

Основной тестовый цикл

Разберёмся подробнее

```
real_t s291(struct args_t * func_args) {  
    initialise_arrays(__func__);  
    gettimeofday(&func_args->t1, NULL);  
    int im1;  
    for (int nl = 0; nl < 2*iterations; nl++) {  
        im1 = LEN_1D-1;  
        for (int i = 0; i < LEN_1D; i++) {  
            a[i] = (b[i] + b[im1]) * (real_t).5;  
            im1 = i;  
        }  
        dummy(a, b, c, d, e, aa, bb, cc, 0.);  
    }  
    gettimeofday(&func_args->t2, NULL);  
    return calc_checksum(__func__);  
}
```

Многократное
тесты для
статистики

выполнение
для
составления

Разберёмся подробнее

```
real_t s291(struct args_t * func_args) {  
    initialise_arrays(__func__);  
    gettimeofday(&func_args->t1, NULL);  
    int im1;  
    for (int nl = 0; nl < 2*iterations; nl++) {  
        im1 = LEN_1D-1;  
        for (int i = 0; i < LEN_1D; i++) {  
            a[i] = (b[i] + b[im1]) * (real_t).5;  
            im1 = i;  
        }  
        dummy(a, b, c, d, e, aa, bb, cc, 0.);  
    }  
    gettimeofday(&func_args->t2, NULL);  
    return calc_checksum(__func__);  
}
```

Переменная состояния

Разберёмся подробнее

```
real_t s291(struct args_t * func_args) {  
    initialise_arrays(__func__);  
    gettimeofday(&func_args->t1, NULL);  
    int im1;  
    for (int nl = 0; nl < 2*iterations; nl++) {  
        im1 = LEN_1D-1;  
        for (int i = 0; i < LEN_1D; i++) {  
            a[i] = (b[i] + b[im1]) * (real_t).5;  
            im1 = i;  
        }  
        dummy(a, b, c, d, e, aa, bb, cc, 0.);  
    }  
    gettimeofday(&func_args->t2, NULL);  
    return calc_checksum(__func__);  
}
```

На первой итерации
используется для
обращения к последнему
элементу массива

Разберёмся подробнее

```
real_t s291(struct args_t * func_args) {  
    initialise_arrays(__func__);  
    gettimeofday(&func_args->t1, NULL);  
    int im1;  
    for (int nl = 0; nl < 2*iterations; nl++) {  
        im1 = LEN_1D-1;  
        for (int i = 0; i < LEN_1D; i++) {  
            a[i] = (b[i] + b[im1]) * (real_t).5;  
            im1 = i;  
        }  
        dummy(a, b, c, d, e, aa, bb, cc, 0.);  
    }  
    gettimeofday(&func_args->t2, NULL);  
    return calc_checksum(__func__);  
}
```

На каждой последующей итерации равна предыдущему значению индукционной переменной

Разберёмся подробнее

```
real_t s292(struct args_t * func_args) {
    initialise_arrays(__func__);
    gettimeofday(&func_args->t1, NULL);
    int im1, im2;
    for (int nl = 0; nl < iterations; nl++) {
        im1 = LEN_1D-1;
        im2 = LEN_1D-2;
        for (int i = 0; i < LEN_1D; i++) {
            a[i] = (b[i] + b[im1] + b[im2]) * (real_t).333;
            im2 = im1;
            im1 = i;
        }
        dummy(a, b, c, d, e, aa, bb, cc, 0.);
    }
    gettimeofday(&func_args->t2, NULL);
    return calc_checksum(__func__);
}
```

Второй тест аналогичен первому, но содержит две переменные состояния

Как это векторизовать?

```
real_t s291(struct args_t * func_args) {
    initialise_arrays(__func__);
    gettimeofday(&func_args->t1, NULL);
    int im1;
    for (int nl = 0; nl < 2*iterations; nl++) {
        im1 = LEN_1D-1;
        a[0] = (b[0] + b[im1]) * (real_t).5;
        im1 = 0;
        for (int i = 1; i < LEN_1D; i++) {
            a[i] = (b[i] + b[im1]) * (real_t).5;
            im1 = i;
        }
        dummy(a, b, c, d, e, aa, bb, cc, 0.);
    }
    gettimeofday(&func_args->t2, NULL);
    return calc_checksum(__func__);
}
```

Если вынести за пределы цикла первую итерацию, в которой переменная состояния не зависит от индукционной переменной, то цикл векторизуется

Промежуточное представление

Промежуточное представление (IR) — это специальный язык, используемый внутри компилятора для совершения архитектурно независимых оптимизаций.

- Все программы сначала компилируются в IR;
- Использование специального языка упрощает оптимизации и позволяет переиспользовать одни и те же оптимизации для разных языков и архитектур.

s291 в промежуточном представлении

5:

; preds = %0, %5

```
%6 = phi i64 [ 0, %0 ], [ %16, %5 ]
%7 = phi i64 [ 31999, %0 ], [ %6, %5 ]
%8 = getelementptr inbounds nuw float, ptr @b, i64 %6
%9 = load float, ptr %8, align 4, !tbaa !9
%10 = and i64 %7, 4294967295
%11 = getelementptr inbounds nuw float, ptr @b, i64 %10
%12 = load float, ptr %11, align 4, !tbaa !9
%13 = fadd fast float %12, %9
%14 = fmul fast float %13, 5.000000e-01
%15 = getelementptr inbounds nuw float, ptr @a, i64 %6
store float %14, ptr %15, align 4, !tbaa !9
%16 = add nuw nsw i64 %6, 1
%17 = icmp eq i64 %16, 32000
br i1 %17, label %2, label %5, !llvm.loop !13
```

Индукционная переменная

s291 в промежуточном представлении

5:

; preds = %0, %5

Переменная состояния

```
%6 = phi i64 [ 0, %0 ], [ %16, %5 ]
%7 = phi i64 [ 31999, %0 ], [ %6, %5 ]
%8 = getelementptr inbounds nuw float, ptr @b, i64 %6
%9 = load float, ptr %8, align 4, !tbaa !9
%10 = and i64 %7, 4294967295
%11 = getelementptr inbounds nuw float, ptr @b, i64 %10
%12 = load float, ptr %11, align 4, !tbaa !9
%13 = fadd fast float %12, %9
%14 = fmul fast float %13, 5.000000e-01
%15 = getelementptr inbounds nuw float, ptr @a, i64 %6
store float %14, ptr %15, align 4, !tbaa !9
%16 = add nuw nsw i64 %6, 1
%17 = icmp eq i64 %16, 32000
br i1 %17, label %2, label %5, !llvm.loop !13
```


s291 в промежуточном представлении

5:

; preds = %0, %5

```
%6 = phi i64 [ 0, %0 ], [ %16, %5 ]
%7 = phi i64 [ 31999, %0 ], [ %6, %5 ]
%8 = getelementptr inbounds nuw float, ptr @b, i64 %6
%9 = load float, ptr %8, align 4, !tbaa !9
%10 = and i64 %7, 4294967295
%11 = getelementptr inbounds nuw float, ptr @b, i64 %10
%12 = load float, ptr %11, align 4, !tbaa !9
%13 = fadd fast float %12, %9
%14 = fmul fast float %13, 5.000000e-01
%15 = getelementptr inbounds nuw float, ptr @a, i64 %6
store float %14, ptr %15, align 4, !tbaa !9
%16 = add nuw nsw i64 %6, 1
%17 = icmp eq i64 %16, 32000
br i1 %17, label %2, label %5, !llvm.loop !13
```

Напрямую зависит от
индукционной переменной

Проблема

Требуется определять наличие переменной состояния в цикле

Решение

Считать, что переменная состояния присутствует, если переменная в фи-узле берёт своё значение напрямую из индукционной переменной

Ещё одна проблема

Это не будет работать для циклов такого вида:

```
real_t s299() {  
    initialise_arrays(__func__);  
    int im1;    im1 = LEN_1D - 1;  
    for (int i = 0; i < LEN_1D; i++) {  
        a[i] = (b[i] + im1) * (real_t).5;  
        im1 = i - 1 + i * 2;  
    }  
    dummy(a, b, c, d, e, aa, bb, cc, 0.);  
    return calc_checksum(__func__);  
}
```

Ещё одна проблема

Это не будет работать для циклов такого вида:

```
real_t s299() {  
    initialise_arrays(__func__);  
    int im1;    im1 = LEN_1D - 1;  
    for (int i = 0; i < LEN_1D; i++) {  
        a[i] = (b[i] + im1) * (real_t) .5;  
        im1 = i - 1 + i * 2;  
    }  
    dummy(a, b, c, d, e, aa, bb, cc, 0.);  
    return calc_checksum(__func__);  
}
```

Переменная состояния
используется в выражении
(не индексирует массив)

s299 в промежуточном представлении

```
5:                                     ; preds = %0, %5
%6 = phi i64 [ 0, %0 ], [ %17, %5 ]
%7 = phi i32 [ 31999, %0 ], [ %18, %5 ]
%8 = getelementptr inbounds nuw float, ptr @b, i64 %6
%9 = load float, ptr %8, align 4, !tbaa !9
%10 = sitofp i32 %7 to float
%11 = fadd fast float %9, %10
%12 = fmul fast float %11, 5.000000e-01
%13 = getelementptr inbounds nuw float, ptr @a, i64 %6
store float %12, ptr %13, align 4, !tbaa !9
%14 = add nuw i64 %6, 4294967295
%15 = shl nuw nsw i64 %6, 1
%16 = add i64 %14, %15
%17 = add nuw nsw i64 %6, 1
%18 = trunc i64 %16 to i32
%19 = icmp eq i64 %17, 32000
br i1 %19, label %2, label %5, !llvm.loop !23
```

Переменная состояния
получает значение не из
индукционной переменной

s299 в промежуточном представлении

```
5:                                     ; preds = %0, %5
%6 = phi i64 [ 0, %0 ], [ %17, %5 ]
%7 = phi i32 [ 31999, %0 ], [ %18, %5 ]
%8 = getelementptr inbounds nuw float, ptr @b, i64 %6
%9 = load float, ptr %8, align 4, !tbaa !9
%10 = sitofp i32 %7 to float
%11 = fadd fast float %9, %10
%12 = fmul fast float %11, 5.000000e-01
%13 = getelementptr inbounds nuw float, ptr @a, i64 %6
store float %12, ptr %13, align 4, !tbaa !9
%14 = add nuw i64 %6, 4294967295
%15 = shl nuw nsw i64 %6, 1
%16 = add i64 %14, %15
%17 = add nuw nsw i64 %6, 1
%18 = trunc i64 %16 to i32
%19 = icmp eq i64 %17, 32000
br i1 %19, label %2, label %5, !llvm.loop !23
```

Но всё ещё зависит от
индукционной переменной
через последовательность
инструкций

Решение

Просматривать последовательность инструкций, из которых переменная получает своё значение, предполагая, что в этой последовательности содержится индукционная переменная

LoopPeelWithStatePass::run

```
PreservedAnalyses LoopPeelWithStatePass::run(Loop &L, LoopAnalysisManager &AM,
                                              LoopStandardAnalysisResults &AR,
                                              LPMUpdater &U) {
    if (!L.isLoopSimplifyForm() || !L.getExitingBlock())
        return PreservedAnalyses::all();

    if (!hasStateVariables(L))
        return PreservedAnalyses::all();

    ValueToValueMapTy VM;
    if (!peelLoop(&L, 1, false, &AR.LI, &AR.SE, AR.DT, &AR.AC, true, VM))
        return PreservedAnalyses::all();

    return PreservedAnalyses::none();
}
```

LoopPeelWithStatePass::run

```
PreservedAnalyses LoopPeelWithStatePass::run(Loop &L, LoopAnalysisManager &AM,  
                                              LoopStandardAnalysisResults &AR,  
                                              LPMUpdater &U) {  
    if (!L.isLoopSimplifyForm() || !L.getExitingBlock())  
        return PreservedAnalyses::all();  
  
    if (!hasStateVariables(L))  
        return PreservedAnalyses::all();  
  
    ValueToValueMapTy VM;  
    if (!peelLoop(&L, 1, false, &AR.LI, &AR.SE, AR.DT, &AR.AC, true, VM))  
        return PreservedAnalyses::all();  
  
    return PreservedAnalyses::none();  
}
```

Пасс обрабатывает циклы

LoopPeelWithStatePass::run

```
PreservedAnalyses LoopPeelWithStatePass::run(Loop &L, LoopAnalysisManager &AM,  
                                              LoopStandardAnalysisResults &AR,  
                                              LPMUpdater &U) {  
    if (!L.isLoopSimplifyForm() || !L.getExitingBlock())  
        return PreservedAnalyses::all();  
  
    if (!hasStateVariables(L))  
        return PreservedAnalyses::all();  
  
    ValueToValueMapTy VM;  
    if (!peelLoop(&L, 1, false, &AR.LI, &AR.SE, AR.DT, &AR.AC, true, VM))  
        return PreservedAnalyses::all();  
  
    return PreservedAnalyses::none();  
}
```

Условие, без которого
пилинг невозможен или
небезопасен

LoopPeelWithStatePass::run

```
PreservedAnalyses LoopPeelWithStatePass::run(Loop &L, LoopAnalysisManager &AM,  
                                              LoopStandardAnalysisResults &AR,  
                                              LPMUpdater &U) {  
    if (!L.isLoopSimplifyForm() || !L.getExitingBlock())  
        return PreservedAnalyses::all();  
  
    if (!hasStateVariables(L))  
        return PreservedAnalyses::all();  
  
    ValueToValueMapTy VM;  
    if (!peelLoop(&L, 1, false, &AR.LI, &AR.SE, AR.DT, &AR.AC, true, VM))  
        return PreservedAnalyses::all();  
  
    return PreservedAnalyses::none();  
}
```

Проверка наличия
переменных состояния

LoopPeelWithStatePass::hasStateVariables

```
bool LoopPeelWithStatePass::hasStateVariables (Loop &L) {
    BasicBlock *Header = L.getHeader();
    BasicBlock *Latch = L.getLoopLatch();

    Value *IndVar = L.getCanonicalInductionVariable();

    for (PHINode &Phi : Header->phis()) {
        if (Phi.getBasicBlockIndex(Latch) < 0)
            continue;

        Value *LatchValue = Phi.getIncomingValueForBlock(Latch);

        if (!(&Phi == IndVar) && isDerivedFromIndVar(LatchValue, L))
            return true;
    }

    return false;
}
```

LoopPeelWithStatePass::hasStateVariables

```
bool LoopPeelWithStatePass::hasStateVariables (Loop &L) {  
    BasicBlock *Header = L.getHeader();  
    BasicBlock *Latch = L.getLoopLatch();  
  
    Value *IndVar = L.getCanonicalInductionVariable();  
  
    for (PHINode &Phi : Header->phis()) {  
        if (Phi.getBasicBlockIndex(Latch) < 0)  
            continue;  
  
        Value *LatchValue = Phi.getIncomingValueForBlock(Latch);  
  
        if (!(&Phi == IndVar) && isDerivedFromIndVar(LatchValue, L))  
            return true;  
    }  
  
    return false;  
}
```

Header — блок, который содержит фи-узлы, выбирающие значения в зависимости от предыдущего блока

LoopPeelWithStatePass::hasStateVariables

```
bool LoopPeelWithStatePass::hasStateVariables (Loop &L) {
    BasicBlock *Header = L.getHeader();
    BasicBlock *Latch = L.getLoopLatch();

    Value *IndVar = L.getCanonicalInductionVariable();

    for (PHINode &Phi : Header->phis()) {
        if (Phi.getBasicBlockIndex(Latch) < 0)
            continue;

        Value *LatchValue = Phi.getIncomingValueForBlock(Latch);

        if (!(&Phi == IndVar) && isDerivedFromIndVar(LatchValue, L))
            return true;
    }

    return false;
}
```

Latch — блок, в котором обновляются значения некоторых переменных, например индукционной

LoopPeelWithStatePass::hasStateVariables

```
bool LoopPeelWithStatePass::hasStateVariables (Loop &L) {  
    BasicBlock *Header = L.getHeader();  
    BasicBlock *Latch = L.getLoopLatch();  
  
    Value *IndVar = L.getCanonicalInductionVariable();  
  
    for (PHINode &Phi : Header->phis()) {  
        if (Phi.getBasicBlockIndex(Latch) < 0)  
            continue;  
  
        Value *LatchValue = Phi.getIncomingValueForBlock(Latch);  
  
        if (!(&Phi == IndVar) && isDerivedFromIndVar(LatchValue, L))  
            return true;  
    }  
  
    return false;  
}
```

В исследуемых циклах Header и Latch представлены одним блоком

LoopPeelWithStatePass::hasStateVariables

```
bool LoopPeelWithStatePass::hasStateVariables (Loop &L) {
    BasicBlock *Header = L.getHeader();
    BasicBlock *Latch = L.getLoopLatch();

    Value *IndVar = L.getCanonicalInductionVariable();

    for (PHINode &Phi : Header->phis()) {
        if (Phi.getBasicBlockIndex(Latch) < 0)
            continue;

        Value *LatchValue = Phi.getIncomingValueForBlock(Latch);

        if (!(&Phi == IndVar) && isDerivedFromIndVar(LatchValue, L))
            return true;
    }

    return false;
}
```

Получение значения канонической
индукционной переменной цикла

LoopPeelWithStatePass::hasStateVariables

```
bool LoopPeelWithStatePass::hasStateVariables (Loop &L) {
    BasicBlock *Header = L.getHeader();
    BasicBlock *Latch = L.getLoopLatch();

    Value *IndVar = L.getCanonicalInductionVariable();

    for (PHINode &Phi : Header->phis()) {
        if (Phi.getBasicBlockIndex(Latch) < 0)
            continue;

        Value *LatchValue = Phi.getIncomingValueForBlock(Latch);

        if (!(&Phi == IndVar) && isDerivedFromIndVar(LatchValue, L))
            return true;
    }

    return false;
}
```

Цикл проходится по всем фи-узлам блока header

LoopPeelWithStatePass::hasStateVariables

```
bool LoopPeelWithStatePass::hasStateVariables (Loop &L) {
    BasicBlock *Header = L.getHeader();
    BasicBlock *Latch = L.getLoopLatch();

    Value *IndVar = L.getCanonicalInductionVariable();

    for (PHINode &Phi : Header->phis()) {
        if (Phi.getBasicBlockIndex(Latch) < 0)
            continue;

        Value *LatchValue = Phi.getIncomingValueForBlock(Latch);

        if (!(&Phi == IndVar) && isDerivedFromIndVar(LatchValue, L))
            return true;
    }

    return false;
}
```

Цикл пропускает значения, которые не обновляются на каждой итерации

LoopPeelWithStatePass::hasStateVariables

```
bool LoopPeelWithStatePass::hasStateVariables (Loop &L) {
    BasicBlock *Header = L.getHeader();
    BasicBlock *Latch = L.getLoopLatch();

    Value *IndVar = L.getCanonicalInductionVariable();

    for (PHINode &Phi : Header->phis()) {
        if (Phi.getBasicBlockIndex(Latch) < 0)
            continue;

        Value *LatchValue = Phi.getIncomingValueForBlock(Latch);

        if (!(&Phi == IndVar) && isDerivedFromIndVar(LatchValue, L))
            return true;
    }

    return false;
}
```

Значение, которое фи-узел выбирает,
если предыдущим блоком был latch

LoopPeelWithStatePass::hasStateVariables

```
bool LoopPeelWithStatePass::hasStateVariables (Loop &L) {  
    BasicBlock *Header = L.getHeader();  
    BasicBlock *Latch = L.getLoopLatch();  
  
    Value *IndVar = L.getCanonicalInductionVariable();  
  
    for (PHINode &Phi : Header->phis()) {  
        if (Phi.getBasicBlockIndex(Latch) < 0)  
            continue;  
  
        Value *LatchValue = Phi.getIncomingValueForBlock(Latch);  
  
        if (!(&Phi == IndVar) && isDerivedFromIndVar(LatchValue, L))  
            return true;  
    }  
  
    return false;  
}
```

Проверка зависимости переменной от индукционной, только если это не сама индукционная переменная

LoopPeelWithStatePass::isDerivedFromIndVar

```
bool LoopPeelWithStatePass::isDerivedFromIndVar(Value *DerivedValue, Loop &L) {
    Value *IndVar = L.getCanonicalInductionVariable();
    if (!IndVar || !DerivedValue)
        return false;

    if (DerivedValue == IndVar)
        return true;

    Value *V = DerivedValue;
    while (auto *CI = dyn_cast<Instruction>(V)) {
        V = CI->getOperand(0);
        if (!V)
            return false;
        if (V == IndVar)
            return true;
    }

    return false;
}
```

LoopPeelWithStatePass::isDerivedFromIndVar

```
bool LoopPeelWithStatePass::isDerivedFromIndVar(Value *DerivedValue, Loop &L) {
    Value *IndVar = L.getCanonicalInductionVariable();
    if (!IndVar || !DerivedValue)
        return false;

    if (DerivedValue == IndVar)
        return true;

    Value *V = DerivedValue;
    while (auto *CI = dyn_cast<Instruction>(V)) {
        V = CI-&gtgetOperand(0);
        if (!V)
            return false;
        if (V == IndVar)
            return true;
    }

    return false;
}
```

Оптимистичная проверка, получает ли переменная состояния своё значение напрямую из индукционной переменной

LoopPeelWithStatePass::isDerivedFromIndVar

```
bool LoopPeelWithStatePass::isDerivedFromIndVar(Value *DerivedValue, Loop &L) {
    Value *IndVar = L.getCanonicalInductionVariable();
    if (!IndVar || !DerivedValue)
        return false;

    if (DerivedValue == IndVar)
        return true;

    Value *V = DerivedValue;
    while (auto *CI = dyn_cast<Instruction>(V)) {
        V = CI-&gtgetOperand(0);
        if (!V)
            return false;
        if (V == IndVar)
            return true;
    }

    return false;
}
```

«Раскрутка» последовательности
инструкций



LoopPeelWithStatePass::isDerivedFromIndVar

```
bool LoopPeelWithStatePass::isDerivedFromIndVar(Value *DerivedValue, Loop &L) {
    Value *IndVar = L.getCanonicalInductionVariable();
    if (!IndVar || !DerivedValue)
        return false;

    if (DerivedValue == IndVar)
        return true;

    Value *V = DerivedValue;
    while (auto *CI = dyn_cast<Instruction>(V)) {
        V = CI-&gtgetOperand(0);
        if (!V)
            return false;
        if (V == IndVar)
            return true;
    }

    return false;
}
```

Если в последовательности встретилась
индукционная переменная, значит
зависимость есть

LoopPeelWithStatePass::run

```
PreservedAnalyses LoopPeelWithStatePass::run(Loop &L, LoopAnalysisManager &AM,  
                                              LoopStandardAnalysisResults &AR,  
                                              LPMUpdater &U) {  
    if (!L.isLoopSimplifyForm() || !L.getExitingBlock())  
        return PreservedAnalyses::all();  
  
    if (!hasStateVariables(L))  
        return PreservedAnalyses::all();  
  
    ValueToValueMapTy VM;  
    if (!peelLoop(&L, 1, false, &AR.LI, &AR.SE, AR.DT, &AR.AC, true, VM))  
        return PreservedAnalyses::all();  
  
    return PreservedAnalyses::none();  
}
```

Если переменная состояния
была найдена, то происходит
вынос первой итерации цикла

Компиляция тестов

Условия компиляции	Опции компиляции
clang version 22.0.0git (без векторизации)	<code>-static -O3 -fstrict-aliasing -march=rv64gcvb -ffast-math -fno-vectorize</code>
clang version 22.0.0git (векторизация)	<code>-static -O3 -fstrict-aliasing -march=rv64gcvb -ffast-math -fvectorize -fslp-vectorize</code>
clang version 22.0.0git (векторизация) + LoopPeelWithStatePass	<code>-static -O3 -fstrict-aliasing -march=rv64gcvb -ffast-math -fpass-plugin=LoopPeelWithState.so -fvectorize -fslp-vectorize</code>

Эксперименты

- Эксперименты проводились на кластере, содержащем 2 узла, построенных на базе **Banana Pi BPI-F3**;
- Вычисления в циклах производились над типом данных **float**

Banana Pi BPI-F3

CPU

- SpacemiT K1
- 8 ядер RV64GCVB

RAM

- 4 GiB LPDDR4

RVV 1.0

- VLEN 256

Результаты

Обозначения, используемые в таблицах:

- **Theoretical S** – теоретическое ускорение, полученное на основе анализа промежуточного представления и равное размеру вектора в промежуточном представлении. Значение, равное единице, означает то, что цикл не был векторизован;
- **Theoretical S w plug** – то же, что и **Theoretical S**, но при компиляции с пассом **LoopPeelWithState**;
- **Clang** – ускорение, полученное при компиляции с помощью **clang version 22.0.0git**;
- **Clang w plug** – то же, что и **Clang**, но компиляция была произведена с применением пасса **LoopPeelWithState**.

	s000	s111	s1111	s112	s1112	s113	s1113	s114	s115	s1115	s116	s118	s119	s1119	s121	s122	s123	s124	s125	s126	s127	s128	s131	s132	s141	s151	s152	s161	s1161	s162	s171
Theoretical S	4.0	4.0	4.0	4.0	4.0	4.0	1.0	1.0	4.0	4.0	1.0	4.0	4.0	4.0	4.0	4.0	1.0	4.0	8.0	1.0	4.0	4.0	4.0	4.0	1.0	4.0	4.0	1.0	1.0	4.0	4.0
Theoretical S w plug	4.0	4.0	4.0	4.0	4.0	4.0	1.0	1.0	4.0	4.0	1.0	4.0	4.0	4.0	4.0	4.0	1.0	4.0	8.0	1.0	4.0	4.0	4.0	4.0	1.0	4.0	4.0	1.0	1.0	4.0	4.0
Clang	4.8	1.2	4.1	3.2	5.0	4.9	1.0	1.0	3.3	0.3	1.0	0.6	0.7	3.7	3.0	2.2	1.0	4.6	1.6	1.0	2.4	1.8	3.0	5.7	1.0	2.9	4.0	0.9	1.0	3.0	3.1
Clang w plug	4.6	1.2	4.0	3.2	4.9	4.9	1.0	1.1	3.2	0.3	1.0	0.6	1.2	2.2	3.0	2.2	1.0	4.2	1.3	1.0	2.4	1.8	3.0	5.9	1.0	2.9	4.0	0.9	1.0	3.0	3.2

	s256	s257	s258	s261	s271	s272	s273	s274	s275	s2275	s276	s277	s278	s279	s1279	s2710	s2711	s2712	s281	s1281	s291	s292	s293	s2101	s2102	s2111	s311	s31111	s312	s313	s314
Theoretical S	1.0	4.0	1.0	1.0	4.0	4.0	4.0	4.0	1.0	4.0	4.0	1.0	4.0	4.0	4.0	4.0	4.0	4.0	1.0	4.0	1.0	1.0	4.0	4.0	4.0	1.0	4.0	1.0	8.0	4.0	4.0
Theoretical S w plug	1.0	4.0	1.0	1.0	4.0	4.0	4.0	4.0	1.0	4.0	4.0	1.0	4.0	4.0	4.0	4.0	4.0	4.0	1.0	4.0	4.0	4.0	4.0	4.0	4.0	1.0	4.0	1.0	8.0	4.0	4.0
Clang	0.9	0.7	1.0	0.8	4.8	2.6	4.8	4.5	0.8	0.8	1.7	1.0	2.3	1.7	2.6	3.2	4.8	4.6	1.0	2.9	1.0	1.0	9.2	1.1	1.0	1.0	5.4	1.0	4.4	3.4	5.4
Clang w plug	0.8	1.1	1.0	0.7	4.8	2.6	4.5	4.4	0.6	0.8	1.6	1.0	2.3	1.7	2.6	3.3	4.8	4.6	1.0	2.8	6.5	9.0	7.9	1.1	1.0	1.0	5.4	1.0	4.4	3.4	5.4

	s451	s452	s453	s471	s481	s482	s491	s4112	s4113	s4114	s4115	s4116	s4117	s4121	va	vag	vas	vif	vpv	vtv	vpvtv	vpvts	vpvpv	vtvtv	vsumr	vdotr	vbor
Theoretical S	1.0	4.0	4.0	4.0	1.0	1.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	1.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0
Theoretical S w plug	1.0	4.0	4.0	4.0	1.0	1.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	1.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0
Clang	1.0	3.1	5.1	3.1	1.0	1.0	1.8	1.9	1.5	2.1	2.0	2.4	1.9	3.1	1.0	2.5	2.1	7.2	3.1	3.1	3.1	3.5	3.7	3.8	5.4	3.4	8.9
Clang w plug	1.0	3.1	4.8	1.9	1.0	1.0	2.0	1.9	1.5	2.1	2.0	2.4	1.9	3.1	1.1	2.5	2.2	8.4	3.2	3.2	3.1	3.6	3.7	3.8	5.4	3.4	8.9

	s172	s173	s174	s175	s176	s211	s212	s1213	s221	s1221	s222	s231	s232	s1232	s233	s2233	s235	s241	s242	s243	s244	s1244	s2244	s251	s1251	s2251	s3251	s252	s253	s254	s255
Theoretical S	4.0	4.0	4.0	4.0	4.0	1.0	1.0	1.0	1.0	4.0	1.0	1.0	1.0	4.0	1.0	1.0	1.0	1.0	1.0	4.0	1.0	1.0	4.0	4.0	4.0	1.0	4.0	4.0	4.0	4.0	4.0
Theoretical S w plug	4.0	4.0	4.0	4.0	4.0	1.0	1.0	1.0	1.0	4.0	1.0	1.0	1.0	4.0	1.0	1.0	1.0	1.0	1.0	4.0	1.0	1.0	4.0	4.0	4.0	1.0	4.0	4.0	4.0	4.0	4.0
Clang	3.1	3.5	3.5	3.0	3.5	1.2	1.0	1.0	1.0	3.1	1.0	0.9	1.0	0.7	0.7	1.0	0.9	1.0	1.0	2.7	1.0	1.0	3.6	4.1	2.0	1.1	1.6	4.6	5.0	7.3	6.2
Clang w plug	3.2	3.4	3.4	3.0	3.6	1.2	1.0	1.0	1.0	3.2	1.0	0.9	1.0	0.7	0.7	1.0	0.8	1.0	1.0	2.7	1.0	1.0	3.2	3.9	1.9	1.0	1.5	4.8	4.5	7.5	6.2

	s315	s316	s317	s318	s319	s3110	s13110	s3111	s3112	s3113	s321	s322	s323	s331	s332	s341	s342	s343	s351	s1351	s352	s353	s421	s1421	s422	s423	s424	s431	s441	s442	s443
Theoretical S	1.0	4.0	8.0	1.0	4.0	1.0	1.0	4.0	1.0	4.0	1.0	1.0	1.0	8.0	4.0	1.0	1.0	1.0	2.0	4.0	2.0	2.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0
Theoretical S w plug	1.0	4.0	8.0	1.0	4.0	1.0	1.0	4.0	1.0	4.0	1.0	1.0	1.0	8.0	4.0	1.0	1.0	1.0	2.0	4.0	2.0	2.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0
Clang	1.0	4.7	8.9	1.0	4.1	1.0	1.0	9.2	1.0	9.2	1.0	1.0	1.1	6.6	8.6	1.0	1.0	0.9	1.0	3.0	1.6	1.2	2.7	3.6	2.6	4.8	2.9	3.1	2.6	1.6	3.8
Clang w plug	1.0	4.7	8.9	1.0	3.9	1.0	1.0	9.2	1.0	9.2	1.0	1.0	0.8	6.6	8.3	1.0	1.0	0.7	1.0	3.0	1.6	1.2	2.6	3.0	2.6	4.8	2.4	3.2	2.6	1.7	3.7

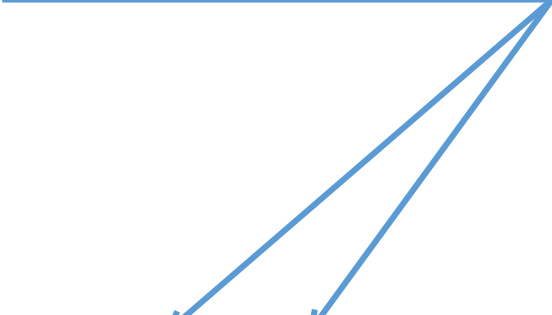
Результаты

Использование пасса позволило успешно векторизовать циклы с сопутствующим увеличением производительности

	s256	s257	s258	s261	s271	s272	s273	s274	s275	s2275	s276	s277	s278	s279	s1279	s2710	s2711	s2712	s281	s1281	s291	s292	s293	s2101	s2102	s2111	s311	s31111	s312	s313	s314
Theoretical S	1.0	4.0	1.0	1.0	4.0	4.0	4.0	4.0	1.0	4.0	4.0	1.0	4.0	4.0	4.0	4.0	4.0	4.0	1.0	4.0	1.0	1.0	4.0	4.0	4.0	1.0	4.0	1.0	8.0	4.0	4.0
Theoretical S w plug	1.0	4.0	1.0	1.0	4.0	4.0	4.0	4.0	1.0	4.0	4.0	1.0	4.0	4.0	4.0	4.0	4.0	4.0	1.0	4.0	4.0	4.0	4.0	4.0	4.0	1.0	4.0	1.0	8.0	4.0	4.0
Clang	0.9	0.7	1.0	0.8	4.8	2.6	4.8	4.5	0.8	0.8	1.7	1.0	2.3	1.7	2.6	3.2	4.8	4.6	1.0	2.9	1.0	1.0	9.2	1.1	1.0	1.0	5.4	1.0	4.4	3.4	5.4
Clang w plug	0.8	1.1	1.0	0.7	4.8	2.6	4.5	4.4	0.6	0.8	1.6	1.0	2.3	1.7	2.6	3.3	4.8	4.6	1.0	2.8	6.5	9.0	7.9	1.1	1.0	1.0	5.4	1.0	4.4	3.4	5.4

Дополнительные проблемы векторизации

Успешная векторизация не всегда приводит к повышению производительности



	s000	s111	s1111	s112	s1112	s113	s1113	s114	s115	s1115	s116	s118	s119	s1119	s121	s122	s123	s124	s125	s126	s127	s128	s131	s132	s141	s151	s152	s161	s1161	s162	s171
Theoretical S	4.0	4.0	4.0	4.0	4.0	4.0	1.0	1.0	4.0	4.0	1.0	4.0	4.0	4.0	4.0	4.0	1.0	4.0	8.0	1.0	4.0	4.0	4.0	4.0	1.0	4.0	4.0	1.0	1.0	4.0	4.0
Theoretical S w plug	4.0	4.0	4.0	4.0	4.0	4.0	1.0	1.0	4.0	4.0	1.0	4.0	4.0	4.0	4.0	4.0	1.0	4.0	8.0	1.0	4.0	4.0	4.0	4.0	1.0	4.0	4.0	1.0	1.0	4.0	4.0
Clang	4.8	1.2	4.1	3.2	5.0	4.9	1.0	1.0	3.3	0.3	1.0	0.6	0.7	3.7	3.0	2.2	1.0	4.6	1.6	1.0	2.4	1.8	3.0	5.7	1.0	2.9	4.0	0.9	1.0	3.0	3.1
Clang w plug	4.6	1.2	4.0	3.2	4.9	4.9	1.0	1.1	3.2	0.3	1.0	0.6	1.2	2.2	3.0	2.2	1.0	4.2	1.3	1.0	2.4	1.8	3.0	5.9	1.0	2.9	4.0	0.9	1.0	3.0	3.2

Заключение и направление дальнейшей работы

- Был реализован оптимизационный пасс, который помогает компилятору векторизовать класс не векторизуемых до этого циклов;
- Эффективность пасса была подтверждена экспериментально;
- В дальнейшем планируется интеграция реализованного алгоритма в пайплайн LLVM для избавления от необходимости в отдельном пассе.



СибГУТИ

СИБИРСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ
ТЕЛЕКОММУНИКАЦИЙ
И ИНФОРМАТИКИ



Кафедра
ВЫЧИСЛИТЕЛЬНЫХ
СИСТЕМ

Спасибо! Вопросы?

Кулик Павел Евгеньевич

студент 4 курса

Институт информатики и вычислительной техники
Сибирский государственный университет телекоммуникаций и информатики

pashakulik58@gmail.com

Научный руководитель — д.т.н. профессор Курносов М.Г.